

[성능 측정 결과]

Random Forest 모델 정확도: 0.9567

```
[[1354  16]
 [  64 415]]
```

	precision	recall	f1-score	support
0	0.95	0.99	0.97	1370
1	0.96	0.87	0.91	479
accuracy			0.96	1849
macro avg	0.96	0.93	0.94	1849
weighted avg	0.96	0.96	0.96	1849

F1 Score : 0.9120879120879121

[정확도를 높일 방법]

일단 모델이 RandomForest를 이용했고, 트리의 개수를 100개로 설정을 했기 때문에, 트리의 개수를 유의미할 정도로 늘린다면 더 높은 정확도와 F1 Score를 얻을 수 있을 것이라고 생각했습니다.

그렇기 때문에 이것을 검증하기 위해서, n_estimators를 우선 300으로 설정하였습니다.

```
# 3. 모델 선언하기 (max_features를 None 또는 's
model_rf = RandomForestClassifier(
    n_estimators=300,          # 트리 개수
```

```
[[1354  16]
 [  63 416]]
```

	precision	recall	f1-score	support
0	0.96	0.99	0.97	1370
1	0.96	0.87	0.91	479
accuracy			0.96	1849
macro avg	0.96	0.93	0.94	1849
weighted avg	0.96	0.96	0.96	1849

F1 Score : 0.9132821075740944

정확도에는 차이는 없으나, 아주 조금의 F1 Score 향상이 있음을 확인하였습니다.

이게 과연 의미가 있는 것인지, 이번에는 트리의 개수를 1000개로 바꾸어 성능 측정을 해보았습니다.

```
# 3. 모델 선언하기 (max_features를 None 또는 'sqrt'로 설정)
model_rf = RandomForestClassifier(
    n_estimators=1000,          # 트리 개수
```

```
[[1355  15]
 [ 66 413]]
      precision    recall  f1-score   support

     0       0.95      0.99      0.97     1370
     1       0.96      0.86      0.91      479

   accuracy          0.96     1849
  macro avg       0.96      0.93      0.94     1849
weighted avg       0.96      0.96      0.96     1849

F1 Score : 0.9106945975744212
```

```
# 3. 모델 선언하기 (max_features를 None 또는 'sqrt', 'log2'로 설정)
model_rf = RandomForestClassifier(
    n_estimators=10000,        # 트리 개수
```

```
[[1354  16]
 [ 67 412]]
      precision    recall  f1-score   support

     0       0.95      0.99      0.97     1370
     1       0.96      0.86      0.91      479

   accuracy          0.96     1849
  macro avg       0.96      0.92      0.94     1849
weighted avg       0.96      0.96      0.95     1849

F1 Score : 0.9084895259095921
```

정확도에는 문제가 없으나, 오히려 F1 Score는 떨어지는 모습을 보여주었습니다. 그래서 트리의 수는 일정 수준 이상이 되면 의미가 없다는 것을 알고(아마 트리가 많아질수록 과적합의 문제가 생기기 때문인 것 같습니다.), 트리의 깊이 설정을 바꿔보려고 했습니다.

트리 수를 300개로 두고, 깊이가 None 설정이 되어있기 때문에, 이것을 조금씩 늘려

300까지 늘려보았습니다.

```
model_rf = RandomForestClassifier(  
    n_estimators=300,          # 트리 개수  
    min_samples_split=2,      # 노드 분할을 위한 최소 샘플 수  
    min_samples_leaf=1,       # 리프 노드가 되기 위한 최소 샘플 수  
    max_features='sqrt',      # 최대 feature 개수 ('sqrt' 권장)  
    max_depth=300,            # 트리의 최대 깊이  
    max_leaf_nodes=None,      # 리프 노드의 최대 개수  
    #random_state=77
```

```
[[1355  15]  
 [ 63 416]]  
      precision    recall  f1-score   support  
  
    0       0.96      0.99      0.97       1370  
    1       0.97      0.87      0.91        479  
  
 accuracy          0.96          1849  
 macro avg       0.96      0.93      0.94       1849  
weighted avg       0.96      0.96      0.96       1849  
  
F1 Score : 0.9142857142857143
```

약간의 성능 향상을 확인할 수 있었습니다. 그러나 정확도에는 유의미한 차이가 없었습니다. 하나의 값을 엄청나게 과하게 설정을 해야, 과적합으로 정확도의 유의미한 상승이 있습니다. 하지만 이렇게 정확도가 상승해도, 결국 F1 Score는 낮아지고, 결국 성능은 좋지 않습니다.

[Panda 전략 활용]

저는 한 번 실행할 때, 시간이 오래 걸리는 걸 좋아하지 않아서, Panda 전략을 활용하였습니다. 그러나 아무리 값을 바꿔도, F1 값은 비슷했습니다. 그래서 이번엔 Cavier 방식을 사용하기로 마음을 바꿨습니다.

[Cavier 전략 활용]

[모델 선언하기]

```
# 기본 모델 선언하기
model_rf3 = RandomForestClassifier(random_state=77)

# 자동으로 탐색할 파라미터 설정
param_grid = [ {'n_estimators':[100,200,300,400,500]}, {'max_depth':[10,20,30,40,50,60,70,80,90,100]}, {'max_leaf_nodes':[10,20,30,40,50,60,70,80,90,100]}]
```

Max_depth의 원소 값을 추가하고, 추가적으로 max_leaf_nodes의 값도 추가하며 최적의 값을 찾도록 했습니다.

실행 중(37분 18초)

오랜 실행 시간 끝에 결과가 나왔습니다.

[최적의 파라미터와 점수 확인]

```
[84] print('최적의 파라미터 값 : ', rf_grid.best_params_)
      print('최고의 점수 : ', rf_grid.best_score_)
```



```
최적의 파라미터 값 : {'max_depth': 10, 'max_leaf_nodes': 100, 'n_estimators': 100}
최고의 점수 : 0.9028624677580211
```

최적의 파라미터 값을 반영해서 값을 다시 수정해보았습니다.

```
[89] # 기본 모델 선언하기
      model_rf3 = RandomForestClassifier(random_state=77)

      # 자동으로 탐색할 파라미터 설정
      param_grid = [ {'n_estimators':[100]}, {'max_depth':[10]}, {'max_leaf_nodes':[100, 200, 300, 400, 500]}]
```

[최적의 파라미터와 점수 확인]

```
[92] print('최적의 파라미터 값 : ', rf_grid.best_params_)
      print('최고의 점수 : ', rf_grid.best_score_)
```



```
최적의 파라미터 값 : {'max_depth': 10, 'max_leaf_nodes': 300, 'n_estimators': 100}
최고의 점수 : 0.9044963283803876
```

최적 값들을 참고하여 좀 더 세부적으로 해보고자 했습니다.

```
[93] # 기본 모델 선언하기
      model_rf3 = RandomForestClassifier(random_state=77)

      # 자동으로 탐색할 파라미터 설정
      param_grid = [ {'n_estimators':[100], 'max_depth':[5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], 'max_leaf_nodes':[300]} ]
```

[최적의 파라미터와 점수 확인]

```
[96] print('최적의 파라미터 값 : ', rf_grid.best_params_)
      print('최고의 점수 : ', rf_grid.best_score_)
```

```
➡ 최적의 파라미터 값 : { 'max_depth': 14, 'max_leaf_nodes': 300, 'n_estimators': 100}
   최고의 점수 : 0.907489680389661
```

아주 약간 상승했음을 알 수 있습니다.

[한계 및 과제 소감]

사실 위의 과정 말고도 정말 많이 값을 바꿔보았습니다. 그러나 들인 시간에 비해 대부분 정확도는 유의미한 차이는 없었고, 이건 어느 전략을 활용해도 같았습니다. 아마 RandomForest를 고집했기 때문인 것 같습니다.

다른 지도 혹은 비지도 학습을 고려하지 않은 것은 아닙니다만, 그렇게 바꿨을 때 아예 변수의 성질 자체가 바뀔 것을 알고 있어서 제가 제대로 해낼 자신이 없었습니다. 그래서 RandomForest 방법은 그대로 하되, 변수 값을 바꿔보거나 최적화 방식을 다르게 해서 유의미한 결과를 얻어보고자 했습니다만 실패했습니다.

과제에서는 실패를 했으나, 수업은 정말 유익했습니다. 저도 인공지능 사이버보안학과이기 때문에 인공지능에 대한 과목을 듣는데, 이렇게 세세하게 풀어서 설명해주신 분이 처음입니다. 제가 만약 인공지능을 이렇게 처음 접했다면, 인공지능을 좀 더 파보지 않을까 아쉬움도 남습니다. 머신러닝 알고리즘 설명만 해주시고 코랩으로 알아서 구현해오라는 분들도 참 많이 계시거든요. 구현하는 방법과 코드 설명, 정말 유익했습니다. 고맙습니다.